

Cybersecurity

Advanced Bash

Lesson 6.1



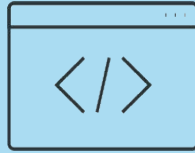
Learning Objectives

By the end of today's class, you will be able to:

- 1 Construct compound commands using `&&`, `|`, and file redirects.
- 2 Create and save alias commands to their `~/.bashrc` file.
- 3 Edit `$PATH` variables to include a custom `~/scripts` directory.
- 4 Create simple bash scripts that consist of a list of commands.



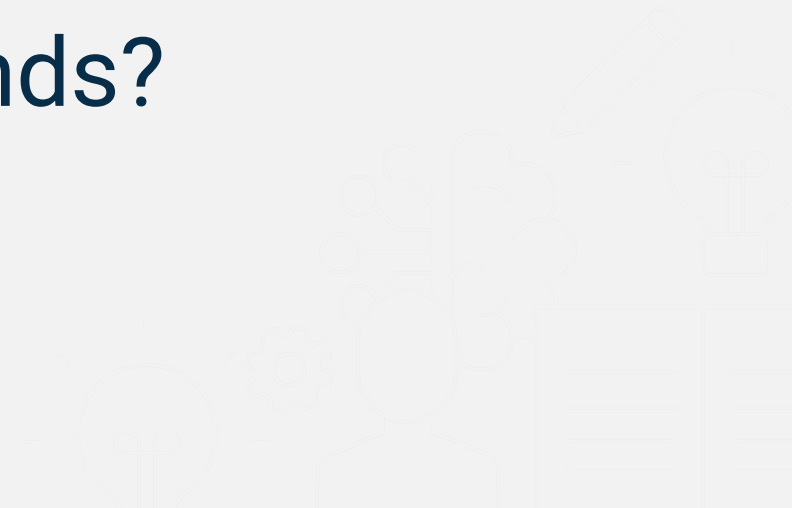
Welcome



Compound **Commands**



Why compound
commands?





Compound Commands

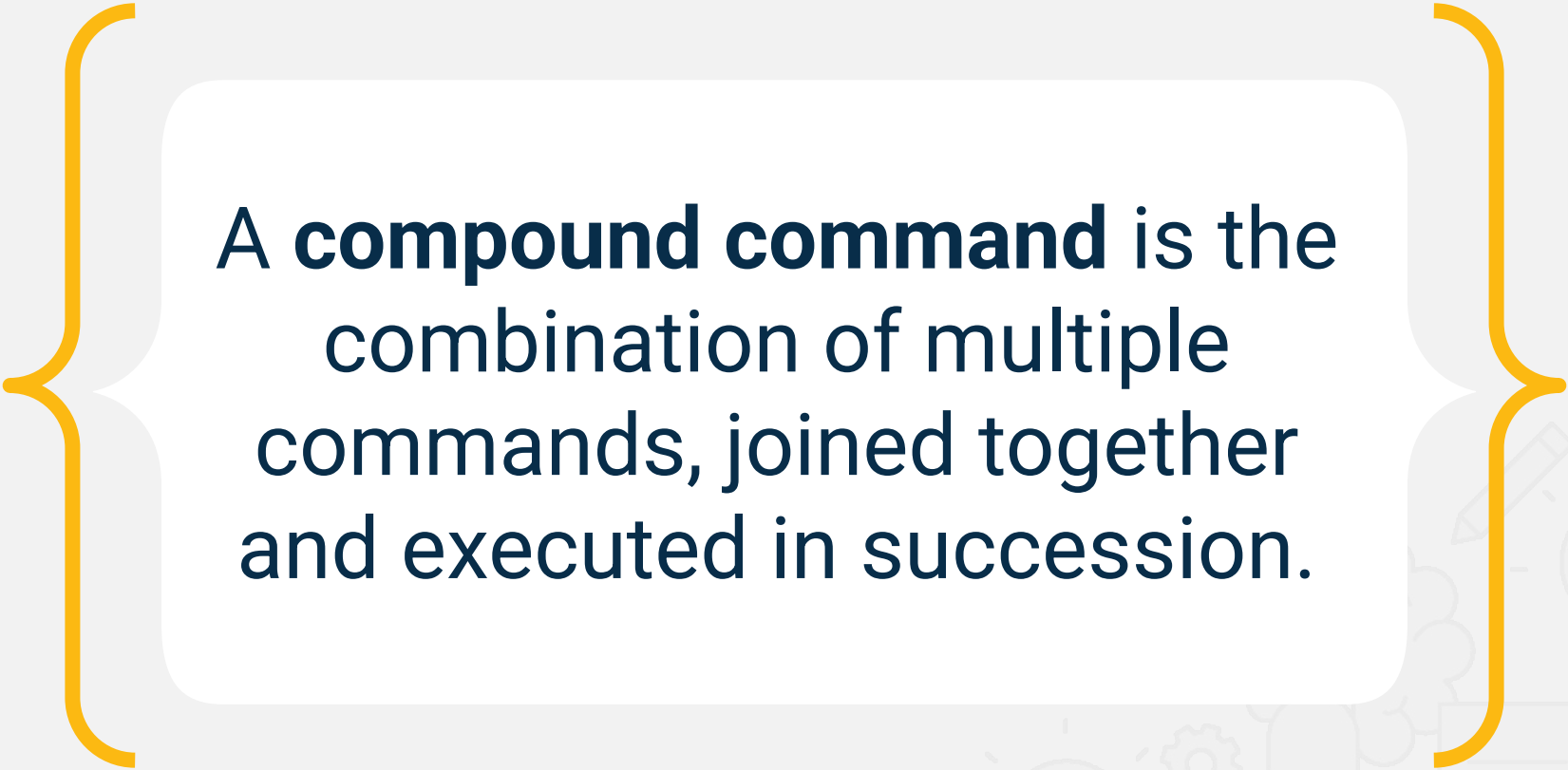
Fundamental IT Skill

Compound Commands:

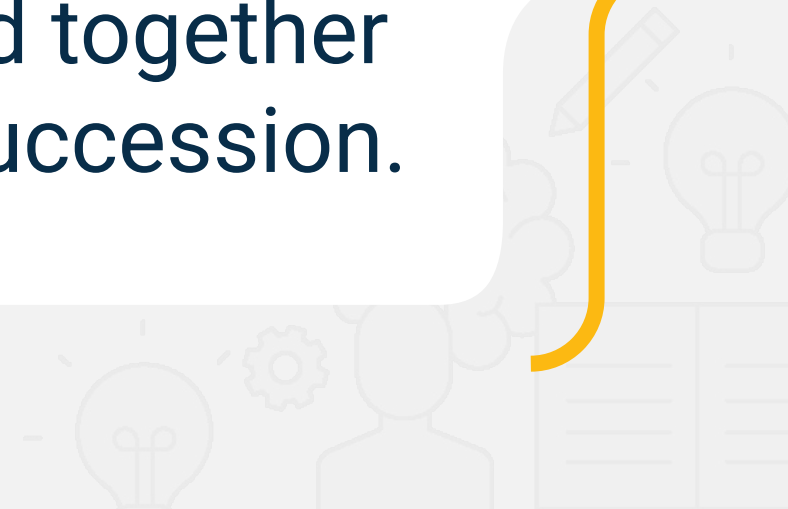
- Navigating directories
- Searching files
- Writing scripts



Linux is widely used in the security field, these skill apply in **many** roles.



A **compound command** is the combination of multiple commands, joined together and executed in succession.





Compound Commands

Syntax Breakdown

```
file $(find / -iname *.txt 2>/dev/null) > ~/Desktop/text_files ; tail ~/Desktop/text_files
```



Searches entire system for `.txt`



Verifies that the files found are text files



Ignores any errors



Creates a list of all found files before saving that list to the desktop



Open the file and print the last 10 lines



Compound Commands

Breaking Down the Command

```
program -options arguments
```

Program	Binary program that you are running
Options	Change the behavior of the program being run in the first part of the command
Arguments	Point the command toward something, such as a directory or file that you want the program to act on



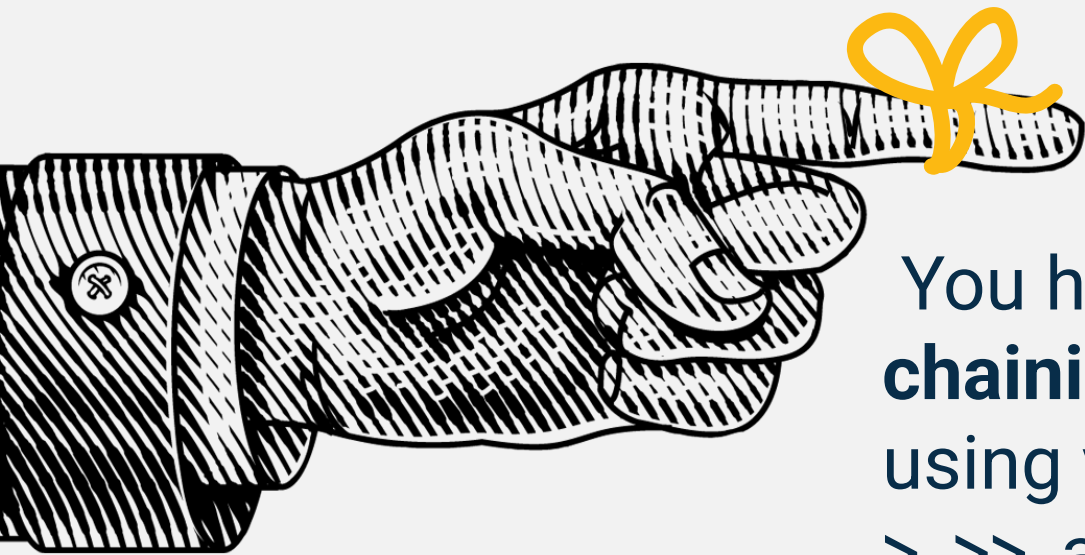
Compound Commands

Typical Format

When chaining commands together, the output of one command becomes the input of the next:

```
program -options arguments | program -options | program -options | program -options
```

Therefore, the **argument** is only needed for the first command in the chain.

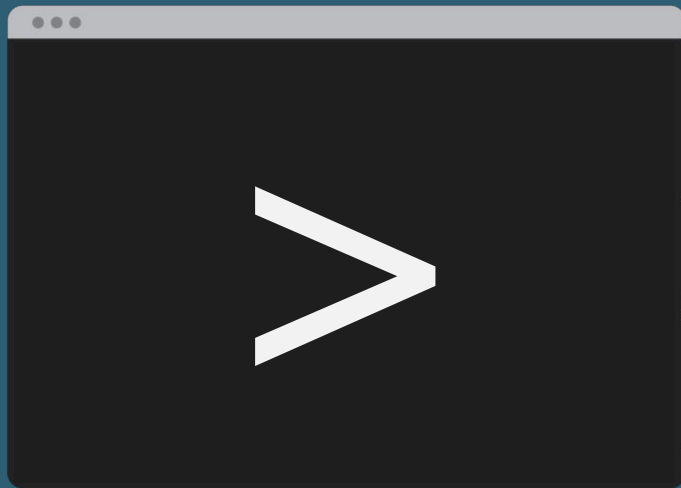


Remember,

You have already been
chaining commands together
using various techniques like
>, >>, and |.

Let's Review

We've already chained commands using `>`, `>>`, and `|`.



Chaining with > and >>

```
ls > list.txt
```

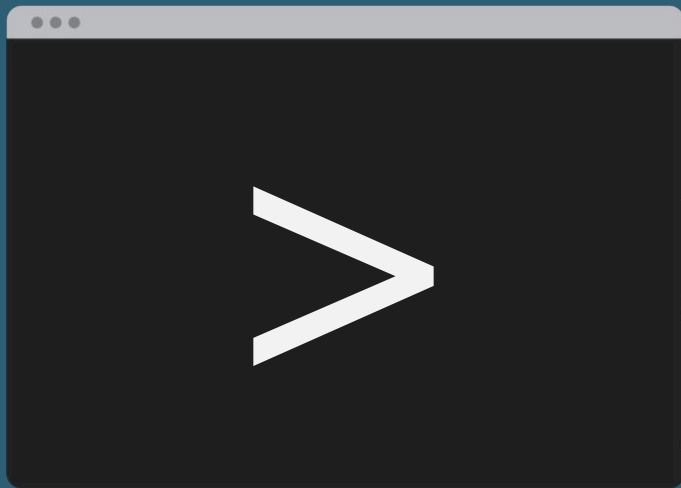
- Takes the output of the `ls` command and sends it into a new file named `list.txt`.
- If the file `list.txt` already exists, it is overwritten.



Chaining with > and >>

```
> list.txt
```

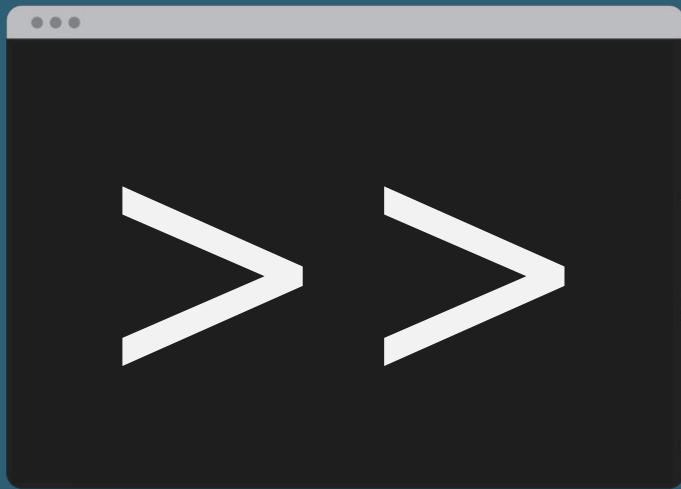
- Without a command in front of >, there is no output to send to the `list.txt` file.
- However, the file is still written, without output, creating a blank file. If the file ``list.txt`` exists, it is overwritten with nothing.



Chaining with > and >>

```
ls >> list.txt
```

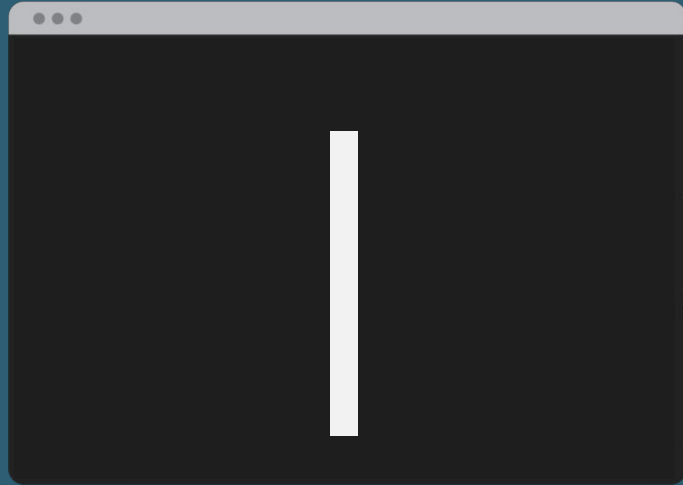
- >> will append the output of the ls.
- If `list.txt` does not exist, it is created.
- Using >> instead of > is always safer, unless you want the file to be overwritten.



Piping with |

- Pipe (|) takes the output of one command and sends it to the input of another command.
- Typically follows this format:

```
program -options arguments |  
program -options | program  
-options | program -options
```





Compound Commands

Piping with |

```
ls -l | grep '.txt'
```



`ls -l` creates a list of files.



`|` pipes the list from `ls` into the command that follows.



`grep` searches the files from `ls` for the string that follows.



`.txt` matches any file that contains `.txt` in the filename.



Review: Piping with |

Some common programs that users will pipe to include:

| **head**

prints only the first 10 lines of output.

| **tail**

prints only the last 10 lines of output.

| **sort**

sorts the output alphabetically.

| **sed**

searches and replaces parts of the output.

| **awk**

displays only specified parts of the output.



Review: Piping with |

Example with `/etc/passwd`

```
cat /etc/passwd | grep sysadmin | awk -F ':' '{print $6}'
```

- `cat /etc/passwd` dumps the contents of `/etc/passwd` to output.
- `|` pipes that output into `grep sysadmin`, displaying lines with `sysadmin`.
- `|` pipes that output into `awk -F ':' '{print $6}'`, printing the 6th field of line.

Combining with ;

- Semicolon (;) is used to run commands back-to-back.
- Each command is running on its own. It is not sending its output to the next command.



Combining with ;

We can use one command:

Rather than running this:

```
$ mkdir dir
$ cd dir
$ touch file
$ ls -l
-rw-r--r-- 1 user user 0
Sep  4 15:33 file
```

```
$ mkdir dir; cd dir; touch
file; ls -l

-rw-r--r-- 1 user user 0 Sep
4 15:33 file
```



Combining with ; Potential Issues

```
mkdir dir; cd dor; touch file; ls -l
```

This command will fail because you are trying to move into the directory **dor**, which has not been created.

Commands **touch** and **ls** will still run.



If a string of commands
separated by ; will run each
command back-to-back, how does
&& affect a chain of commands?

Combining with &&

A better operator to use in the previous case is the &&.

The && will run the next command only if the first command was successful.





Combining with &&

A Better Option

```
mkdir dir && cd dir && touch file && ls -l
```

`cd` would only run if `mkdir` was successful. `touch` would only run if `cd` is successful. `ls` would only run if `touch` is successful.

```
mkdir dir && cd dor && touch file && ls -l
```

Only `mkdir dir` and `cd dor` would run. `cd dor` fails, so `touch` and `ls` are ignored.



Section Summary

We reviewed the following:

>

Creates or writes to existing files with text or output of a command

>>

Appends text or output of a command to a file

|

Pipes the output of one command into another command

;

Chains commands together in succession

&&

Displays only specified parts of the output



Activity:

Compound Commands

Now we can start combining commands together to save us time when working on the command line.

- In this activity, you are working on auditing a new system and would like to simplify your job with automation. You will begin by combining several commands together to make fewer overall commands.

Suggested Time:
15 minutes

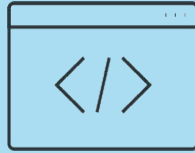


Time's up!
Let's review



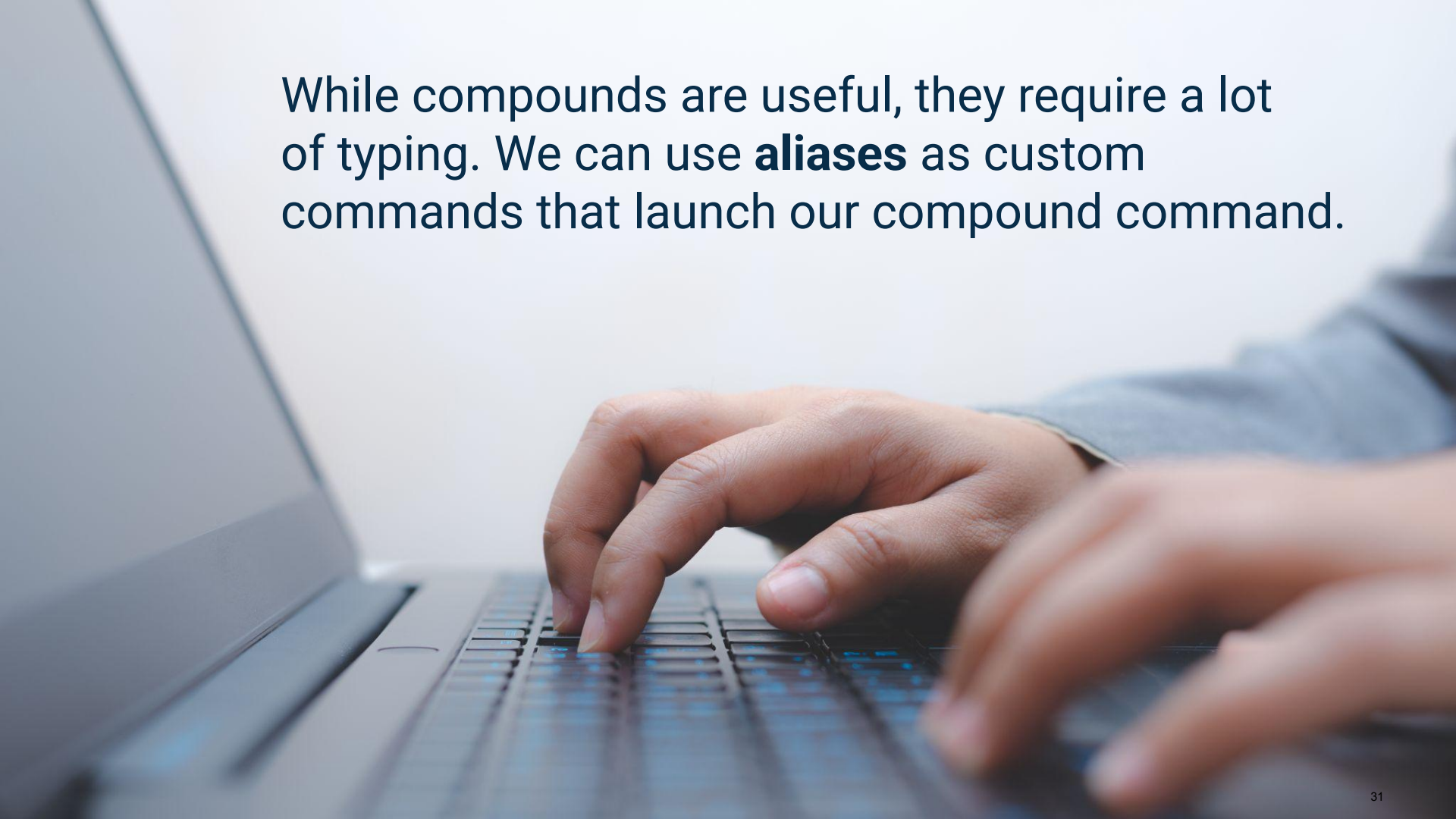
Questions?





Creating Aliases

While compounds are useful, they require a lot of typing. We can use **aliases** as custom commands that launch our compound command.





Aliases

Syntax Breakdown

```
alias lh="ls -lah"
```



`alias` indicates that we are creating an alias.



`lh` is our custom command for storing commands we want to run.

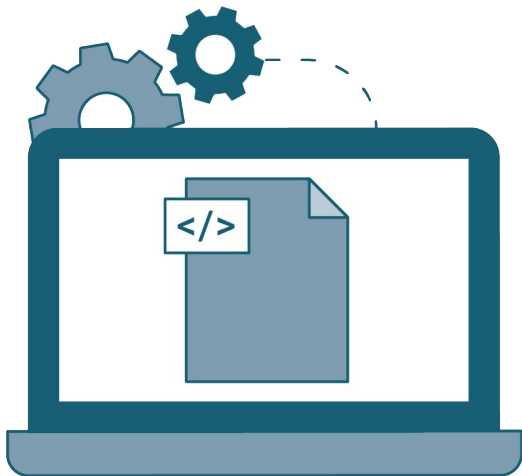


`ls -lah` is the command that will run when we use our alias `lh`.

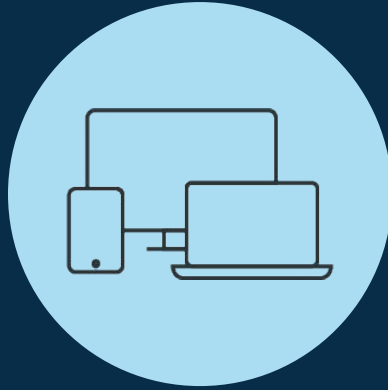


Aliases

Keeping Aliases Across Sessions and Logins



- Store consistent aliases in the `.bashrc` file.
- Add alias definitions in `.bashrc` for easy access.
- Use `source ~/.bashrc` to apply changes.
- `.bashrc` ensures aliases are available in every new terminal session.



Instructor Demonstration

Aliases & `.bashrc`



Activity:

Creating Aliases

In this activity, we will create several aliases and save them to our `~/.bashrc` file so that they will always be available.

Suggested Time:
15 minutes





Time's up!
Let's review

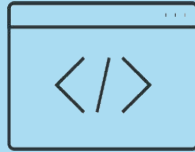


Questions?





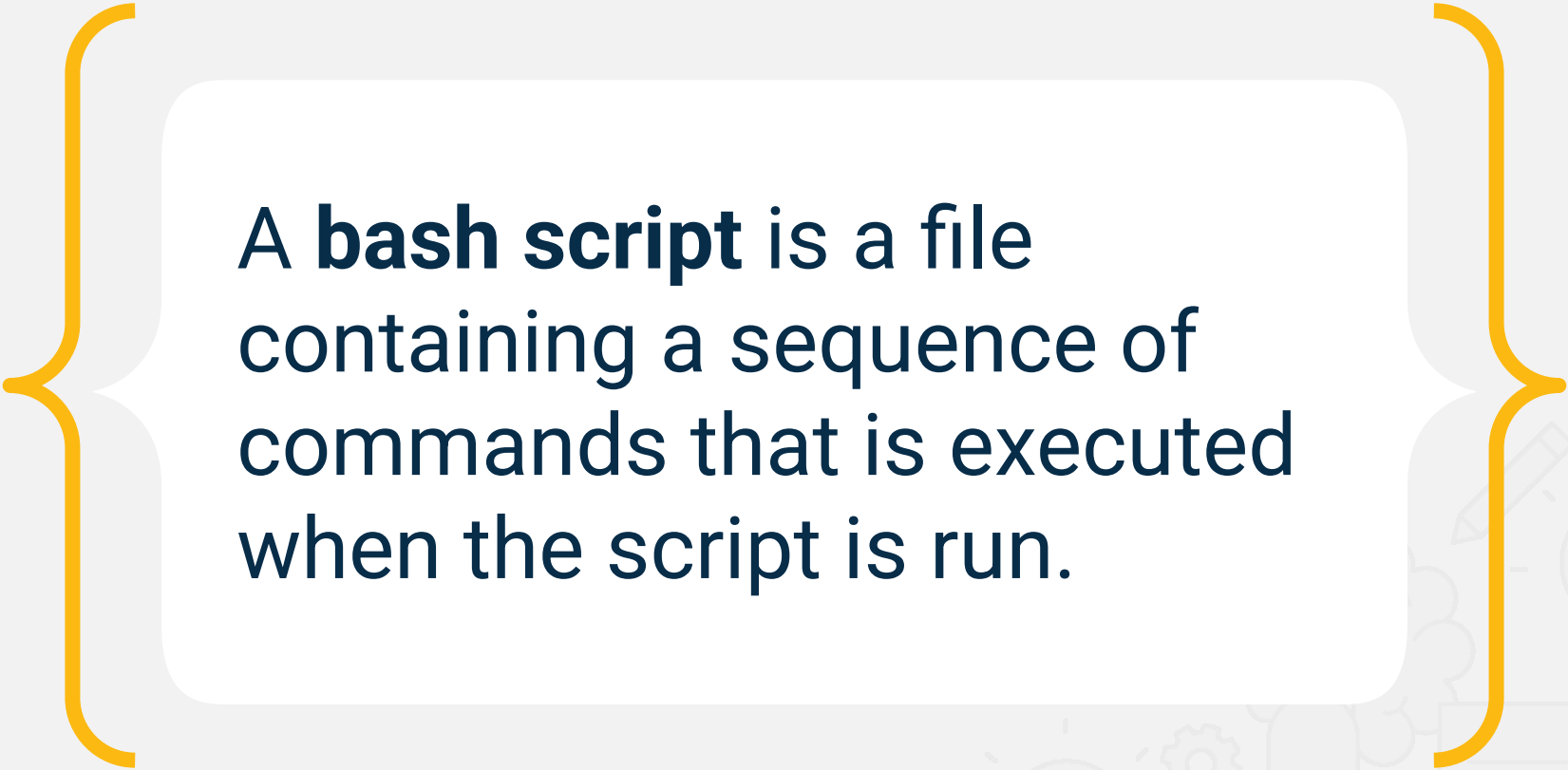
Break
15 mins



Creating **Bash Scripts**

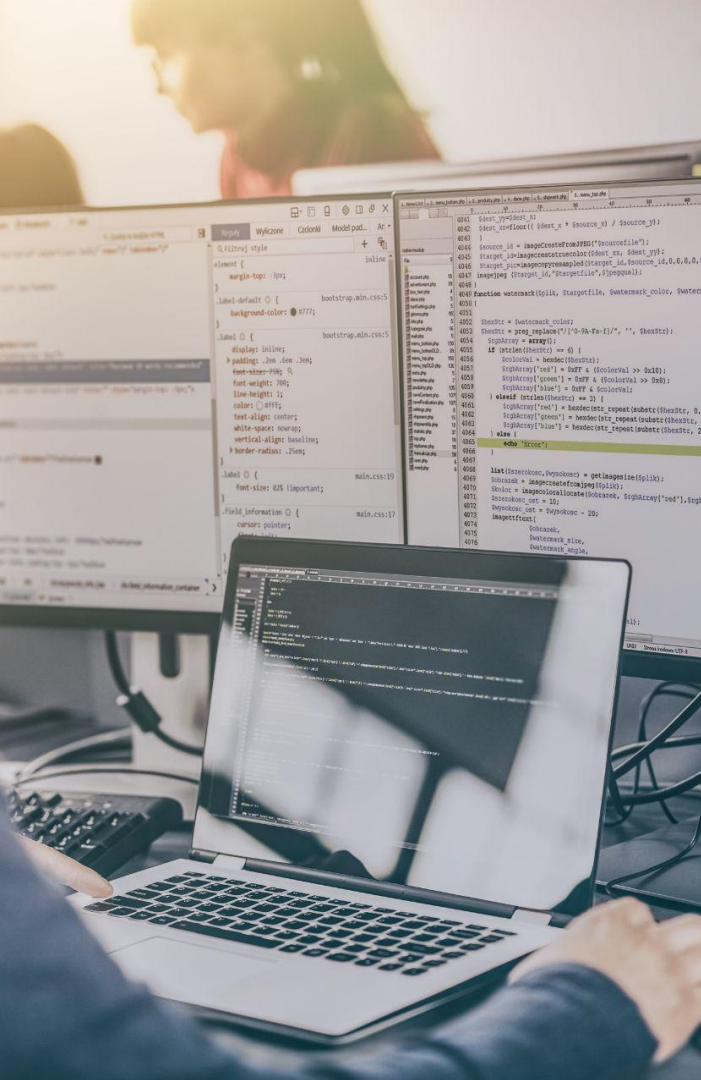


Can anyone define what
a basic **bash script** is?



A **bash script** is a file containing a sequence of commands that is executed when the script is run.





Introduction to Variables

Variables in Computer Programming

- In computer programming, a **variable** is a location that stores some kind of data.
- It's a “**box**” that holds something so that you can refer to it later.
- You can overwrite its contents with new contents.

Variable Demo

In the following demo, we will use:



Basic variables



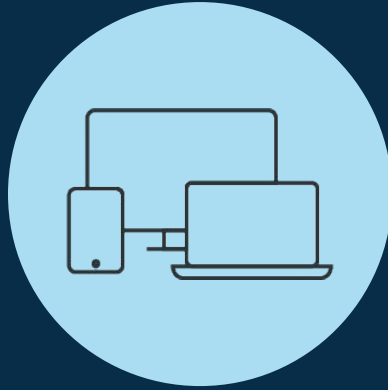
Built-in variables



Command expansion



Variables in scripts



Instructor Demonstration

Variable Demo



Questions?





Activity:

Sysadmin Bash Script

In the previous activity, you created several aliases and saved them to your `~/.bashrc` file. You may have noticed that creating long strings of commands can get a bit cumbersome when working on the command line.

- In this activity, you will create a script that completes several system audit steps automatically.

Suggested Time:
20 minutes

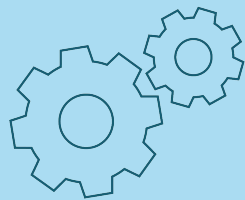


Time's up!
Let's review



Questions?

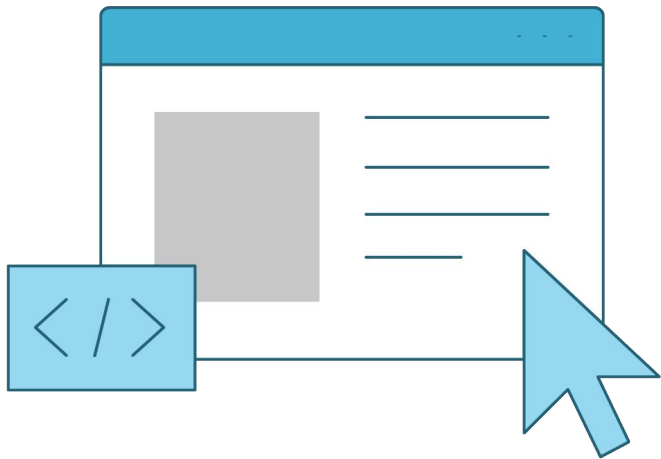




Custom Commands

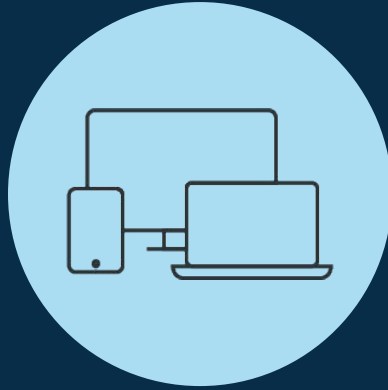


Demonstration Scenario



We will create a custom command that runs our script.

- This requires a bit of knowledge of what happens behind the scenes.
- We'll look at a built-in variable known as the **PATH** variable.



Instructor Demonstration

PATH



Activity:

Custom Commands

We will continue to make our script more complex by adding a few commands from the first exercise.

- You will save this script to a **scripts** directory and add that directory to your **\$PATH** so you can call your script directly on the command line.

Suggested Time:
20 minutes



Time's up!
Let's review



Questions?





Next class

Ifs and Lists



Thank you